

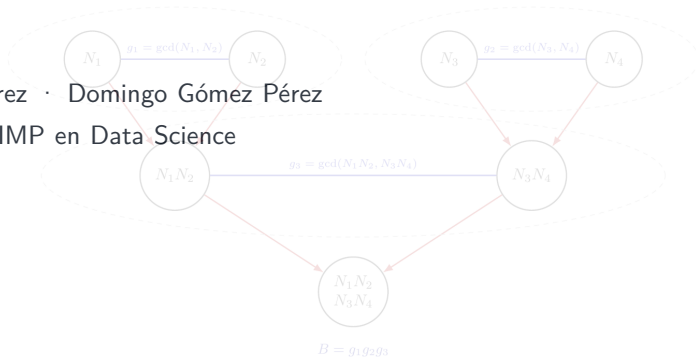
Búsqueda de factores RSA compartidos en Certificate Transparency logs

Un enfoque basado en Binary Tree Batch GCD

David Quinzaños Saiz

Codirectores: Ana I. Gómez Pérez · Domingo Gómez Pérez

Máster Interuniversitario UC–UIMP en Data Science



Una idea central

Si dos claves RSA comparten un primo, dejan de ser seguras.

El reto del TFM es buscar ese fallo en colecciones reales de certificados sin comparar todos los pares de claves.

54.275

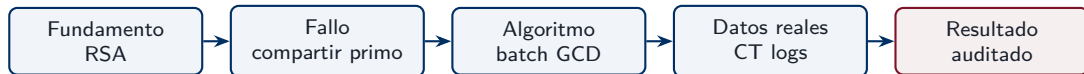
modulos RSA plausibles auditados

23,8 s

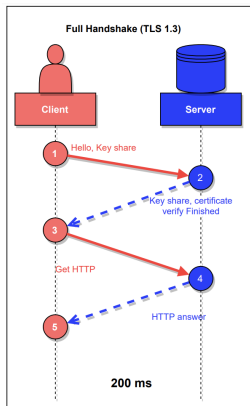
ejecucion real del algoritmo

0

factores compartidos detectados



RSA dentro de la confianza web



Punto de partida

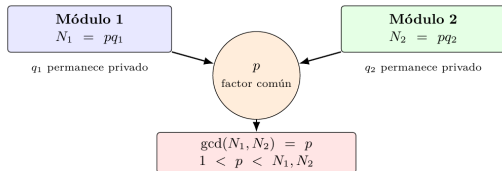
La Web confía en certificados digitales que contienen claves públicas. Una parte relevante de ese ecosistema usa RSA.

- En RSA, la clave pública contiene un módulo $N = p \cdot q$.
- La seguridad práctica depende de que p y q sean secretos, grandes e independientes.
- Un certificado publicado permite auditar el módulo público, no la clave privada.

La vulnerabilidad: compartir un primo

$$N_1 = pq_1, \quad N_2 = pq_2$$

$$\gcd(N_1, N_2) = p$$



Consecuencia

Recuperar p factoriza ambos modulus. No se rompe RSA por factorizar un entero difícil, sino por explotar un fallo de generacion.

En condiciones ideales, esta coincidencia debería ser prácticamente imposible.

Certificate Transparency como fuente de datos



Protecciones de conexión para unican.es

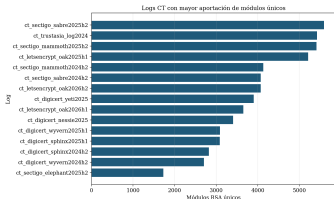


Está conectado de forma segura a este sitio.

Verificado por: Hellenic Academic and Research Institutions CA

Por que CT logs

Son registros publicos, verificables y de solo anexo donde aparecen certificados emitidos por autoridades reconocidas.



- Permiten pasar de un problema teórico a una auditoria empirica.
- Evitan depender solo de escaneos de Internet con certificados internos o de prueba.
- Ofrecen volumen suficiente para que la eficiencia algoritmica importe.

Objetivo y aportaciones

Objetivo general

Estudiar la detección eficiente de factores primos compartidos entre módulos RSA y aplicarla a certificados reales procedentes de Certificate Transparency.

01

revisar batch GCD y vulnerabilidades RSA

02

implementar Binary Tree Batch GCD en C++17/GMP

03

construir un pipeline reproducible sobre ct-archive

La contribución no es solo algorítmica: también incluye extracción, normalización, validación, deduplicación y auditoría de módulos reales.

De la evidencia previa al hueco de este TFM

Trabajo	Escenario	Resultado	Hueco
Heninger et al.	HTTPS/SSH a gran escala	claves factibles	Våge muestra que CT logs son una fuente relevante, pero el análisis masivo es costoso. Pelofske propone un algoritmo mas eficiente. Este TFM conecta ambas lineas.
Våge	CT logs, 159,4 M RSA-2048	8 claves + 2 roSSL	
Pelofske	mejora de batch GCD	Binary Tree Batch GCD	

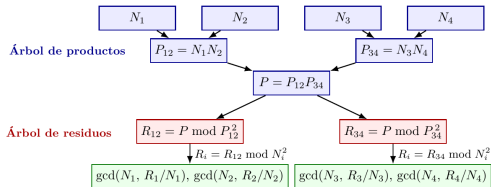
Pregunta guía: ¿puede una implementacion C++/GMP hacer practica esta auditoria sobre datos reales?

El cuello de botella: comparar todos los pares

$\binom{M}{2}$
comparaciones ingenuas

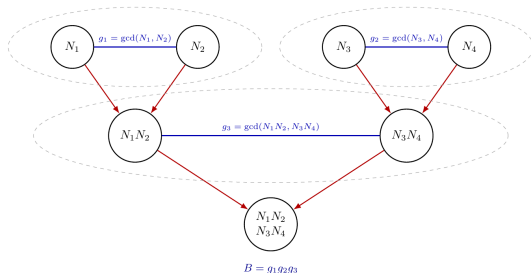
1.472.860.675
pares para 54.275 módulos

$2M - 1$
operaciones GCD
en el enfoque usado



- El metodo clasico usa *product tree* y *remainder tree*.
- El coste practico se concentra en enteros intermedios muy grandes.
- El objetivo es conservar la idea batch, reduciendo trabajo y memoria.

Binary Tree Batch GCD



1. Construye un arbol de productos.
2. Calcula gcd entre hermanos durante la construccion.
3. Agrega los g_j no triviales en $B = \prod g_j$.
4. Enumera $\gcd(N_i, B)$ para recuperar factores.

Idea clave

Evita construir explícitamente el *remainder tree* del enfoque clasico.

Implementacion desarrollada

C++17 + GMP

aritmetica multiprecision optimizada

OpenMP

paralelizacion de la enumeracion final

Liberacion temprana

menor pico de memoria durante el
arbol

Validacion funcional

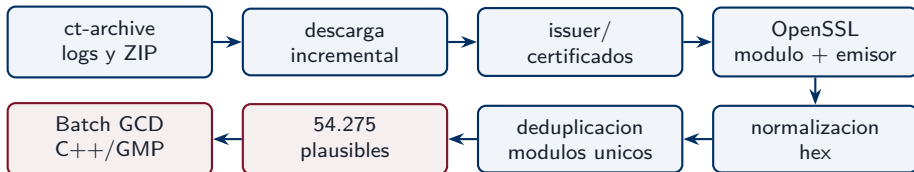
Las tres implementaciones evaluadas detectan los mismos modulos vulnerables en los experimentos sinteticos.

- Referencia: implementaciones Python de Pelofske.
- Comprobacion: divisibilidad exacta $N \bmod p = 0$.
- Salida: factores y modulos afectados.

Decision practica

La mejora se concentra en GMP, gestion de memoria y paralelizacion donde aporta.

Pipeline real desde ct-archive

**54.278**

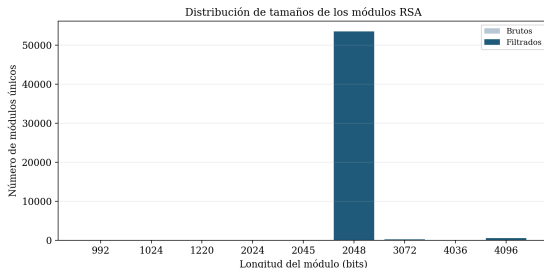
modulos unicos brutos

3

descartados por validacion

20logs con modu-
los extraidos

Un conjunto real, no una entrada artificial

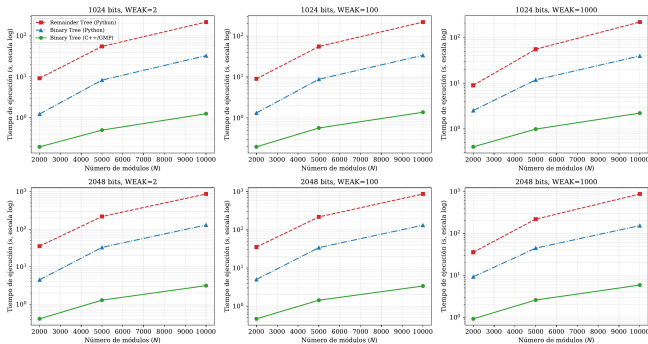


Bits	Modulos plausibles
1024	9
2045	1
2048	53.539
3072	204
4036	1
4096	521

Lectura

Predominan los modulos RSA-2048, pero la muestra conserva diversidad real de tamanos.

Benchmark sintético: tiempo



Orden observado

Remainder Tree Python >
Binary Tree Python >
C++/GMP.

116,5x

speedup medio

C++/GMP vs clasico

275,3x

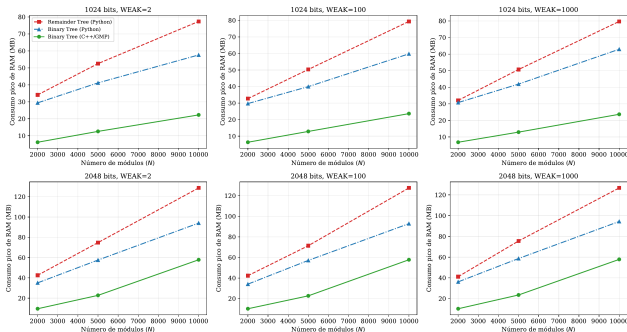
speedup maximo

5,85 s

caso 2048b, N=10000,

WEAK=1000

Benchmark sintético: memoria



128,7 MB

pico Remain-
der Tree Python

94,4 MB

pico Binary Tree Python

57,9 MB

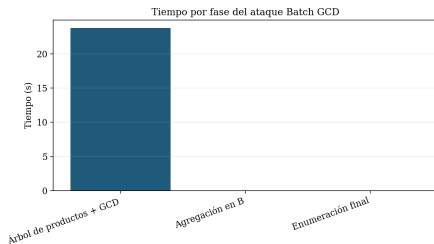
pico C++/GMP

Interpretacion

La liberacion temprana de nodos reduce el pico de memoria y acerca la tecnica a una auditoria practica.

Resultado sobre CT logs

Metrica	Valor
Modulos RSA plausibles	54.275
GCDs no triviales fase 1	0
Claves vulnerables	0
Tiempo total	23,756 s



El cero tambien es informacion.

Interpretacion correcta

No se detectaron factores RSA compartidos en el subconjunto extraido, validado y deduplicado. Esto no demuestra ausencia global en todos los CT logs.

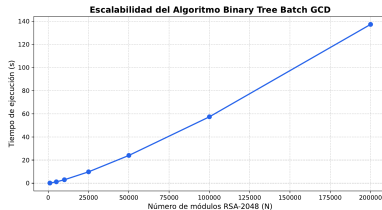
El resultado valida el flujo completo: desde certificados reales hasta una auditoria criptografica ejecutada localmente en segundos.

Conclusiones

1. Binary Tree Batch GCD reduce el coste practico frente al enfoque clasico.
2. La implementacion C++17/GMP mejora de forma decisiva a las referencias Python.
3. El flujo desarrollado permite auditar modulos RSA reales extraidos de CT logs.
4. En 54.275 modulos plausibles no se encontraron factores compartidos.

Limitaciones y futuro

Ampliar a certificados hoja, escalar a mas logs y paralelizar fases todavia secuenciales.



Gracias.